Fraud Detection Pipeline: Change Log

DecodeLabs Data Science | Project 2 | Batch 2026

Supervised Learning · SMOTE, Logistic Regression and Random Forest · Generated 2026-11-14 04:10:57

1. Dataset Overview

Source file	creditcard.csv (Kaggle, "Credit Card Fraud Detection", Université Libre de Bruxelles)
Size	284,807 rows × 31 columns
Columns	Time, V1 to V28 (anonymised PCA components), Amount, Class (1 = fraud)
Class balance	492 frauds in 284,807 rows (0.1727%)
Tool	Python (pandas, scikit-learn, imbalanced-learn) in Jupyter Notebook; exported model and Streamlit dashboard on top
Deliverables	project2_fraud_detection_pipeline.ipynb, dashboard/ (Streamlit app), dashboard/artifacts/rf_fraud_pipeline.joblib, this change log
Data handling	The 150 MB dataset is never committed to git; it exceeds GitHub's 100 MB limit. A 2,473-row demo sample is committed instead.

2. Change Log

ID	Description	Impact	Status
CR001	Removed exact duplicate rows before splitting. An identical transaction sitting in both the training and test sets would inflate the test score, which is a quiet form of leakage.	1,081 duplicate rows removed. 283,726 rows kept; fraud cases 492 to 473.	Resolved
CR002	Checked every column for missing values. No imputation was needed, so none was applied: inventing values would have been a change with no evidence behind it.	0 missing values across 31 columns. Nothing imputed.	Resolved (no action needed)
CR003	Sanity-checked the data: **Class** must be 0 or 1, and **Time** and **Amount** cannot be negative. The notebook asserts these rather than assuming them.	0 violations. Assertions pass before any model is trained.	Resolved (no action needed)
CR004	Kept extreme values instead of capping them. In Project 1 outliers were clipped, but here the fraud rows are themselves the unusual ones, so capping would erase the signal being learned.	0 rows altered. Evidence: about 97% of fraud rows are outliers in at least one V column against about 45% of legitimate rows; a fraud row is extreme in about 12 of the 28 V columns against about 1 for a legitimate row.	Resolved (deliberate)
CR005	Replaced accuracy with Precision, Recall, F1, ROC-AUC and average precision. Accuracy rewards a model for ignoring the rare class.	A baseline that labels everything legitimate scores 99.83% accuracy with 0% recall, so accuracy is not used anywhere in this project.	Resolved

ID	Description	Impact	Status
CR006	Made a stratified 80/20 train/test split before any resampling or scaling, with seed 42. The test set is set aside and not looked at again until the final evaluation.	Train 226,980 rows (378 frauds) | test 56,746 rows (95 frauds).	Resolved
CR007	Applied SMOTE inside an *imblearn* Pipeline, so it is fitted only on the training part of each fold and is skipped automatically at prediction time.	Test rows resampled: 0. Synthetic fraud examples are built from training rows only.	Resolved
CR008	StandardScaler placed inside the Logistic Regression pipeline only, because **Amount** and **Time** are on different scales from **V1** to **V28**. The Random Forest is unscaled, as the brief specifies; trees split on thresholds.	Scaling fitted on training folds only; 0 test statistics used.	Resolved
CR009	Tuned with GridSearchCV, 5-fold stratified cross-validation, refitting on ROC-AUC. The search ran on a stratified 100,000-row sample of the training set to keep run time reasonable, then the winning settings were retrained on the full training set.	Logistic Regression: C=0.01, SMOTE k=5 (CV ROC-AUC 0.9798). Random Forest: max_depth=10, SMOTE k=5 (CV ROC-AUC 0.9833).	Resolved (assumption A3)
CR010	Chose the decision threshold on training data only, using cross-validated out-of-fold probabilities and a rule fixed in advance: the highest recall among thresholds whose precision is at least the precision at 0.50.	At 0.50: precision 0.611, recall 0.844. At 0.40: precision 0.451, recall 0.857. At 0.30: precision 0.275, recall 0.873. Lowering the cut-off bought almost no recall for a large precision loss, so 0.50 was kept. The F1-maximising threshold on training data was 0.81.	Resolved
CR011	Selected the Random Forest as the model to deploy, on test-set evidence: almost all of its alerts are real fraud, so an investigator is not sent on 19 false chases for every real one.	Precision 0.6016, recall 0.8105, F1 0.6906, ROC-AUC 0.9729, average precision 0.7716. Logistic Regression caught slightly more fraud (recall 0.8737) but at precision 0.0534.	Resolved
CR012	Exported the fitted Random Forest pipeline and built a Streamlit dashboard on top of it. The export script rebuilds the model from the dataset with the same split and seed, then refuses to pass unless it reproduces the notebook's test scores.	Model file 1,521 KB (compressed), served by the dashboard. Reproduction check: all 6 checks passed. The 150 MB dataset is not committed; a 2,473-row demo sample is.	Resolved

3. Before vs. After

Metric	Before	After
Rows	`284,807`	`283,726`
Columns	`31 (Time, V1-V28, Amount, Class)`	`31 (unchanged)`
Fraud cases	`492`	`473`
Fraud share	`0.1727%`	`0.1667%`
Missing values	`0`	`0`
Exact duplicate rows	`1,081`	`0`

Metric	Before	After
Rows with capping or imputation	0	0 (deliberate)
Rows used for training	-	226,980
Rows held back for testing	-	56,746
Test rows resampled by SMOTE	-	0

4. Results on the Untouched Test Set

Scores at the default threshold of 0.50, on 56,746 rows the models never saw during training or tuning, containing 95 real frauds. Each test fraud is worth about one percentage point of recall, which is the honest way to read these numbers.

Model	Precision	Recall	F1	ROC-AUC	Avg precision
Baseline (always legitimate)	0.0000	0.0000	0.0000	0.5000	0.0017
Logistic Regression	0.0534	0.8737	0.1007	0.9623	0.6582
Random Forest (deployed)	0.6016	0.8105	0.6906	0.9729	0.7716

> **Why precision is the harder number here.** The test set is imbalanced: only 95 of 56,746 rows are fraud. A false-positive rate of one in a thousand still produces dozens of false alarms, so a model can look excellent on ROC-AUC while most of its alerts are wrong. The Random Forest's average precision of 0.7716 sits far above the 0.17% a random guess would get, which is why it is the model that was deployed.

5. Threshold Decision

Measured on cross-validated out-of-fold predictions for the training set, so the test set had no part in choosing it. The rule was fixed before the numbers were seen: highest recall among thresholds whose precision is at least the precision at 0.50.

Threshold	Precision	Recall	F1
0.50	0.611	0.844	0.709
0.40	0.451	0.857	0.591
0.30	0.275	0.873	0.419

Chosen threshold: **0.50**. At the deployed cut-off the model catches 77 of 95 test frauds, misses 18, and raises 51 false alarms. The threshold that maximised F1 on the training data was 0.81. Applied to the test set once, it gives precision 0.873 and recall 0.726 (F1 0.793), catching 69 and missing 26, with 10 false alarms - against 77 caught, 18 missed and 51 false alarms at 0.50. Moving there trades recall for precision; whether that is worth it depends on what a missed fraud costs. It is reported for transparency and was not used to choose the deployed threshold.

6. Verification Gate

Checks run against the actual deliverable, not against a description of it. The reproduction rows come from rebuilding the model from creditcard.csv with the same split and seed, then comparing with the notebook's recorded scores.

Check	Requirement	Result
Exported model reproduces the notebook	Precision 0.6016 and recall 0.8105 on the test set, with 77 caught, 18 missed, 51 false alarms	**PASS**
No leakage from test to train	0 test rows resampled; scaler and SMOTE fitted inside the pipeline	**PASS**

Check	Requirement	Result
Stratification held	Fraud share of train and test within one fraud case of each other	**PASS**
Threshold chosen without the test set	Rule fixed in advance and applied to out-of-fold training predictions	**PASS**
Accuracy not used as a success metric	Results reported with precision, recall, F1, ROC-AUC and average precision	**PASS**
Model file small enough to commit	Well under GitHub's 100 MB limit, and the 150 MB dataset stays untracked	**PASS (1,521 KB)**
Versions pinned	requirements.txt matches the environment the model was saved in	**PASS**

Reproduction check (test set, threshold 0.50)	Notebook	Rebuilt model	Result
precision	`0.6016`	`0.6016`	**PASS**
recall	`0.8105`	`0.8105`	**PASS**
f1	`0.6906`	`0.6906`	**PASS**
roc_auc	`0.9729`	`0.9729`	**PASS**
avg_precision	`0.7716`	`0.7716`	**PASS**
confusion matrix at 0.50 (caught/missed/false alarms)	`[77, 18, 51]`	`[77, 18, 51]`	**PASS**

Tolerance 0.0005 for the rounded notebook values. The confusion-matrix row checks 77 caught, 18 missed and 51 false alarms exactly.

7. Notes & Assumptions

- **A1.** The data is the Credit Card Fraud Detection dataset from Kaggle. **V1** to **V28** are anonymised components, so no column can be interpreted on its own and no raw card data can be fed to this model. Only **Time** and **Amount** are original, unscaled fields.
- **A2.** Exact duplicate rows are removed before the split so that an identical transaction cannot sit in both parts. This is a judgement call: duplicates may be real repeat transactions, but leaving them risks leakage, and leakage would overstate the model. Removing them is the conservative choice.
- **A3.** The hyperparameter search ran on a 100,000-row stratified sample of the training set because a full search was slow. The winning settings were then retrained on all 226,980 training rows. The test set was never involved.
- **A4.** Outliers were kept, unlike Project 1. Fraud rows are unusual by nature; clipping them would remove the pattern the model is meant to learn.
- **A5.** With only 473 fraud cases in the cleaned data, the metrics are noisy. A single test fraud changes recall by about one percentage point, so small differences between models should not be over-read.
- **A6.** The 0.50 threshold is a business trade-off, not a statistical optimum. It assumes a false alarm costs much less than a missed fraud. A real deployment would set it from the actual costs of investigation and of blocking a customer.

8. What a score means, measured

The forest was trained on a SMOTE-balanced set, so its output ranks transactions but does not behave like a probability. The table puts the model's own average score beside what the true labels say happened, on the untouched test set. A calibrated model would have those two columns agree. This one runs high: its mean score over all rows is 2.5% against a true fraud rate of 0.17%.

Test rows by score	Rows	Frauds	Model's mean score	Actual fraud rate	95% range of the rate
below 0.50	56,618	18	2.3%	0.03%	0.0% to 0.1%
0.50 to 0.90	53	12	62.8%	22.64%	13.5% to 35.5%
0.90 and above	75	65	99.1%	86.67%	77.2% to 92.6%
all rows	56,746	95	2.5%	0.17%	0.1% to 0.2%

The ordering survives: the higher the band, the higher the measured rate, which is why a threshold works at all. At the deployed cut-off, 77 of 128 flagged test transactions were real fraud, so about 60% of alerts are genuine (95% range 51% to 68%). The honest percentage to quote is the measured rate for a group, never a single row's score. Fitting a calibrator (Platt scaling or isotonic regression) would pull the numbers down to reality but would not change the ranking or the 0.50 decision, so it was left out.

9. Limits, Reproducibility and Attribution

- **Limits.** This is an academic demonstration on a public dataset, not a production fraud system. It has not been tested on live data, it has no drift monitoring, no case-management workflow and no fair-lending or adverse-action review, which a real system would require.
- **Reproducibility.** Environment: Python 3.13.9, scikit-learn 1.7.2, imbalanced-learn 0.14.0, pandas 2.3.3, numpy 2.3.5. The pinned versions in dashboard/requirements.txt match the versions the model was saved with, so the pickle cannot silently change behaviour between training and deployment.
- **Attribution.** Dataset: Machine Learning Group, Université Libre de Bruxelles (ULB), published for research on classification of highly imbalanced data.